

言語処理系 第10回
動的メモリ管理と
簡単なレジスタ割り当て

木村啓二

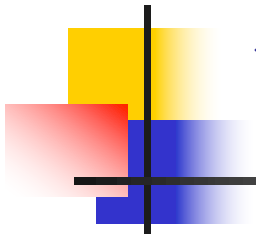
第9回演習問題(1)解答例

- 以下のプログラムを実行した結果表示される値を答えよ。また、その根拠も簡潔に述べよ。

```
int func(int a)
{
    a++;  /* この文の影響がmainにあるかどうか、という問題 */
    return a;
}

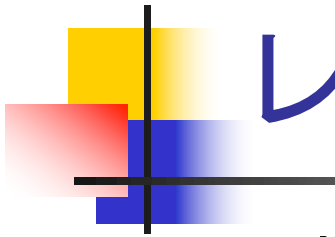
int main()
{
    int t, v = 1;
    t = func(v);
    printf("%d¥n", v+t);
    return 0;
}
```

- 3（理由：引数は値渡し（コピーを呼び出し先に渡す）なので、呼び出し元の関数に影響を与えず、v+tは1+2を計算するため）



第9回演習問題 (2)

- 第7回「静的変数とグローバル変数」のスライドの実行結果を示せ。また、c1()及びc2()それぞれのcountの値とgcounterの値がなぜそのようになったか、理由を簡潔に説明せよ。
- 出力「s1 4, s2 9, g 15」
  各countとgcounterの初期値は0、c1()とc2()はそれぞれ5回、15回呼び出される。countのインクリメントはreturn文の評価後となるため、呼び出し回数より1つ小さな値となる。

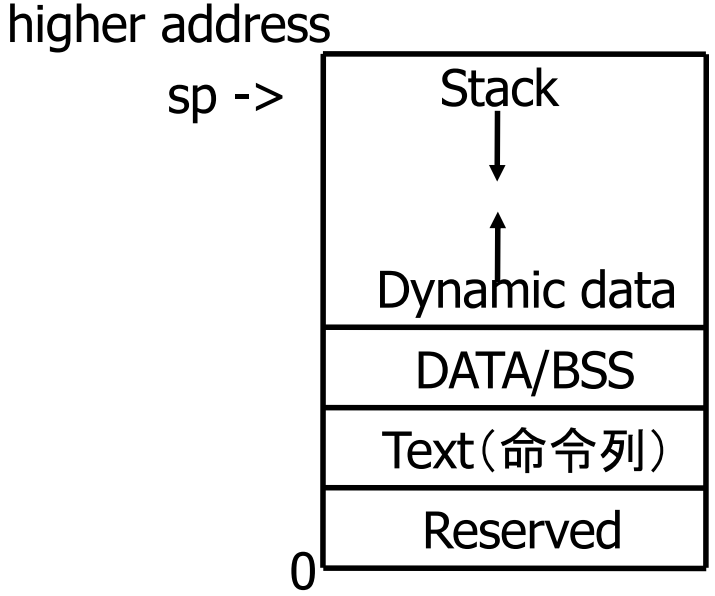


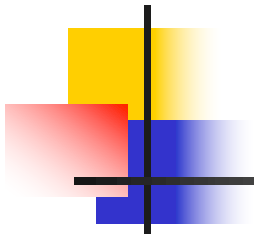
レジスタとメモリ

- いずれもコンピュータの中の記憶素子
- レジスタ
  - 最も高速なメモリ素子
  - 数は少ない
  - プログラムの高速化のためにはレジスタの有効利用が必要
- メモリ（主記憶）
  - プログラムとデータの初期配置場所（ちょっとウソ）
  - 自動変数はスタックフレーム上に確保
  - 静的変数はコンパイル時に決められた領域に固定的に確保
  - 動的確保の変数は？
    - CのmallocやC++のnewで確保されるような領域

広域変数やヒープ

- Cにおける広域（グローバル）変数や静的（スタティック）変数
  - DATAやBSSと呼ばれる特定の領域に確保される
  - 未初期化変数はプログラム実行時に0で初期化される
- 動的に確保されたオブジェクトはヒープ(heap)を利用する
  - CのmallocやJava, C++のnew





ヒープ

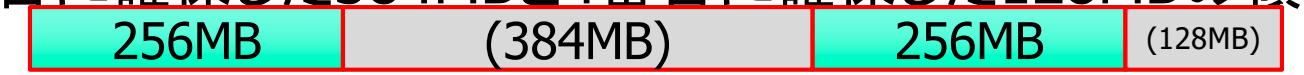
- 動的に確保されるオブジェクト用の領域
- Cならmalloc(), calloc()
  - free()で解放
- C++, Javaならnew
  - C++はdelete
  - Javaは？
- 仮想記憶を利用するOSであれば、ヒープは必要に応じてOSからメモリを割り当てられ拡張する
  - プロセスの実行が終了すると、そのプロセスが利用していたヒープはもちろんTEXT等も含めてメモリ領域はまるごとOSに返却される

ヒープ管理とメモリ断片化 (memory fragmentation)

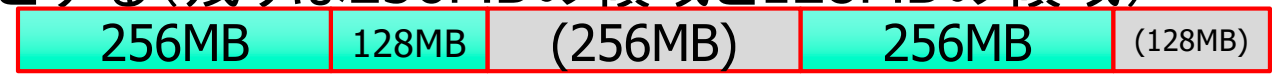
- メモリ確保・解放を繰り返すと、細かい空き領域が生じる
- 例:
  - ヒープ1GBまで利用可能な状態で256MB, 384MB, 256MB, 128MBの領域を順に確保



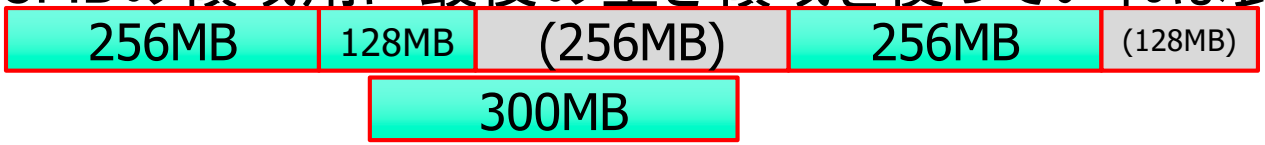
  - 2番目に確保した384MBと4番目に確保した128MBの領域を開放

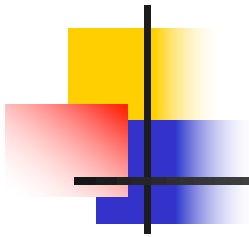


  - 次に128MBの領域を確保
    →最初に見つかった384MBの空き領域のうちの128MBを割り当てるとする(残りは256MBの領域と128MBの領域)



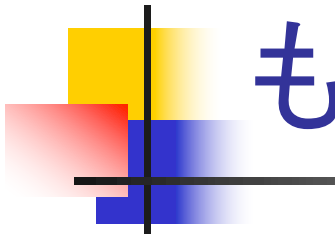
  - 300MBの領域を割り当てようとする
    →連続した300MB以上の領域がないので失敗
    →128MBの領域用に最後の空き領域を使っていれば良かった





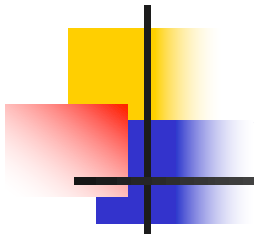
best-fitとfirst-fitアルゴリズム

- 要求されたサイズの空き領域を探すアルゴリズム
  - 空き領域は通常、双方向リストで管理される
  - 空き領域のリストをフリーリストと呼ぶ
- best-fit
  - フリーリストを全て探索し、要求サイズを満たす最小の空き領域を割り当てる
  - ヒープの利用効率は良いが、探索時間が長くなる
- first-fit
  - フリーリストを先頭から探索して、要求サイズを満たす空き領域があればそれを割り当てる
  - 探索時間は短いが、先のスライドの通り空き領域の合計が要求サイズよりも大きいが割り当てられない場面が生じる



もっと効率よくヒープを管理する

- メモリ効率向上と空き領域の探索時間短縮を両立できないか？
- 一般に利用されるヒープ管理方式の基本的な方針
  - サイズ毎にフリーリストを設けて管理する
  - 例：16バイトのリスト、32バイトのリスト、64バイトのリスト…
  - 各サイズのリストをbin（ビン）と呼ぶ
  - 実用的なプログラムでは小さい領域を数多く確保するので、そのようなプログラムに適する方式



演習問題(1)

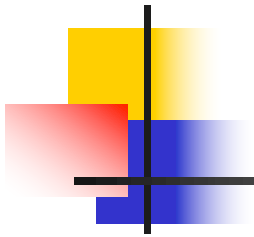
- フリーリストに空き領域が以下の順番で登録されているとする:

  80バイト、30バイト、60バイト、
  50バイト、20バイト

  今、以下の順番で領域確保の要求があった場合、best-fitとfirst-fitの各々でどのように処理されるか:

  60バイト、10バイト、40バイト、70バイト

  - Moodleでは少し形式を変更



レジスタ

- メモリ階層中、最も速いメモリ
  - 演算器に直結する
  - 数は少ない
    - x86-64の整数レジスタは16個
    - RISC系CPUの整数レジスタは通常32個
- なるべくメモリアクセスしないようなコード生成が重要
  - レジスタを有効利用する
    - メモリアクセス（ロード命令・ストア命令）を最小化するべきである
  - とりあえず、使えるだけのレジスタを使い、足りなくなったらスタックに退避する、といった方法でもなんとかなる

レジスタ割り当て

- どの変数に対してレジスタを使い（**配置**し）、それら個々の変数をどのレジスタに**割り当てる**か、を決定する
- レジスタ配置（Register allocation）
  - プログラムの各地点で、どの変数をレジスタに格納するか決める
  - 現在のCPUでは、通常は演算に使用される変数は、遅かれ早かれレジスタに一度格納して利用されることが多い
    - ここで「配置」を使用するのはあまり一般的ではないかも
- レジスタ割り当て（Register assignment）
  - プログラムの各地点で、ある変数をあるレジスタに割り当てる

(1) a = b + c
(2) d = a + b



```
LD R1, b
LD R2, c
ADD R3,R1,R2
ADD R2,R3,R1
ST R2,d
```

(1)でa, b, cはレジスタに配置される
(1)でbはR1に割り当てられる
(1)でcはR2に割り当てられる
(1)でaはR3に割り当てられる
(2)でd, a, bはレジスタに配置される
（cは配置を外される）

単純なレジスタ割り当て手法
前提

- プログラム中の基本ブロック毎に処理する
  - 基本ブロックは途中で分岐等のない文の列
  - 一直線で実行できる
  - 詳しくは笠原先生のパートで
- 以下の2つのデータ構造を導入する
  - 各レジスタのレジスタ記述子（Register descriptor）
    - 対象レジスタに割り当てられている変数を記録する
  - 各変数のアドレス記述子（Address descriptor）
    - 対象変数がどこに格納されているか記録する
      - レジスタ番号やメモリのアドレス
    - シンボルテーブルに配置できる

単純なレジスタ割り当て手法 手順

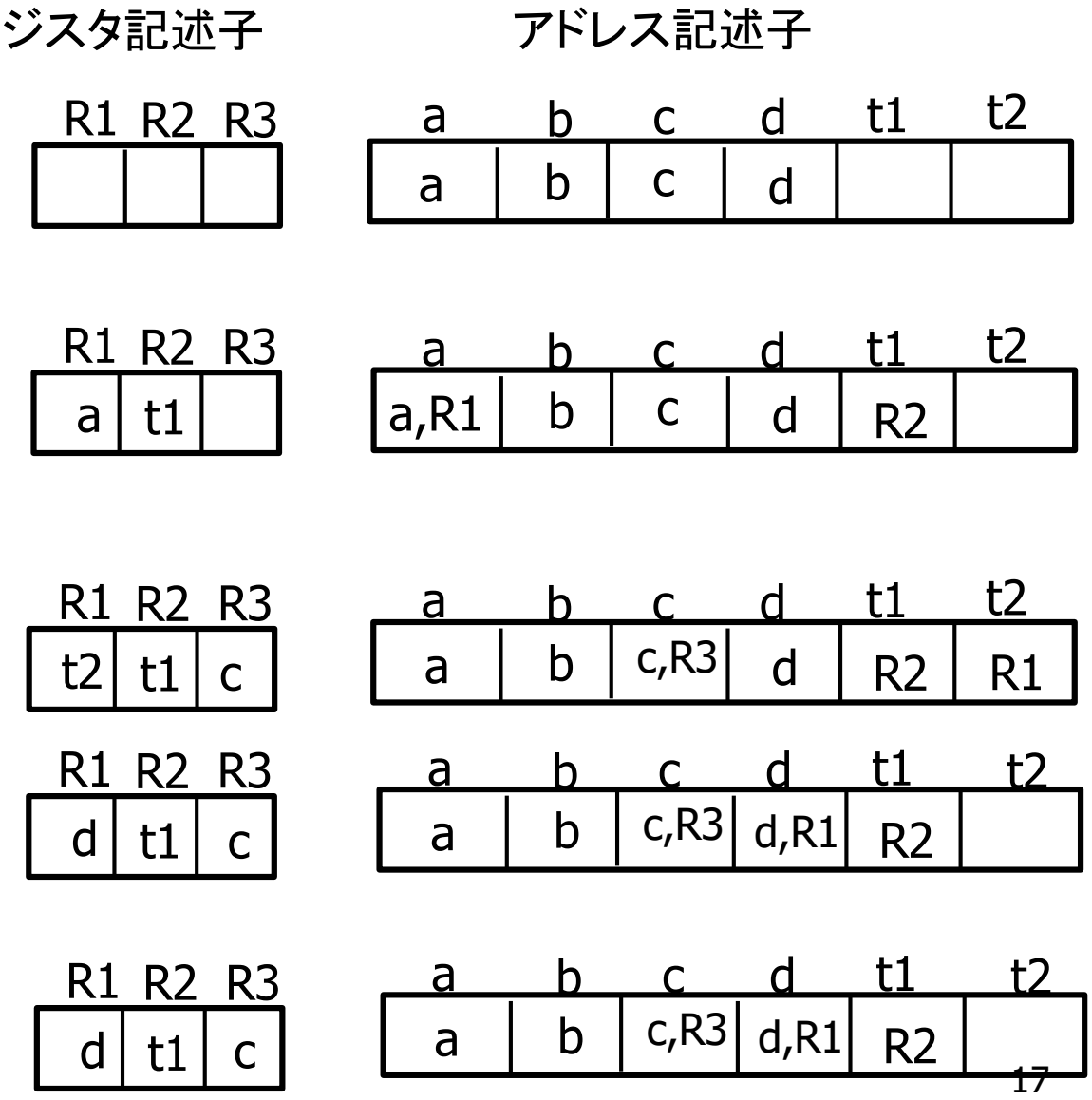
- 中間表現中の各演算に対して以下を処理する
  - 演算のオペランド（変数）がレジスタ記述子に存在しない場合、それを新しい（空き）レジスタに割り当てる
    - 別の値を保持している場合でも、その値がその後使われないなら、古い値を上書きすることでそのレジスタを割り当てる
      - その後使われるかどうかはプログラムの解析により判断できる（授業の範囲外）
    - ソースオペランドの場合は、それを割り当てられたレジスタにロードする
    - 対応するアドレス記述子に割り当てられたレジスタの情報を記録する
  - 空きレジスタが無い場合は？
    - どれか一つ使用中のレジスタを割り当てる
    - 使用中のレジスタ中の値が更新されている場合、それはメモリにストアする（spillする）
  - 基本ブロックの最後まで到達したら、後で使われる可能性のある更新済みのレジスタ中の値をストアする

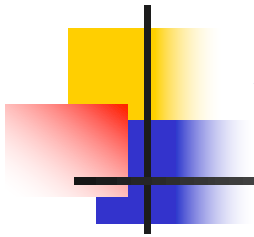
簡単なプログラムによる例

- t1=a-b
  t2=a-c
  d=t1+t2
  - a, b, dは関数局所変数
  - dは基本ブロックの最後まで使われる可能性がある
  - t1, t2は一時変数

簡単なプログラムのレジスタ割り当ての様子

- t1=a-b
  - LD R1,a
    LD R2,b
    SUB R2,R1,R2
- t2=a-c
  - LD R3,c
    SUB R1,R1,R3
- d=t1+t2
  - ADD R1,R2,R1
- (基本ブロック最後)
  - ST R1,d





拡張の方針

- より頻繁に使われる変数を優先的にレジスタに割り当てる
  - 多重ネストループの最内側ループで使用される変数など
- レジスタの数が足りない場合
  - 使用頻度の低い変数をレジスタから追い出す、など
- レジスタ最適化に関しては様々な手法が提案された
  - グラフ彩色の応用等

演習問題(2)

- 「簡単なプログラムのレジスタ割り当ての様子」を参考に、以下のコードのアセンブリを書け（かけ算はmul命令が使える）。なお、レジスタはR1,R2,R3の3つが利用可能である。また、a,b,c,dは後で利用される可能性がある。

```
b=a+b
c=a*b
a=a-c
d=a*c
```